- AI Cache · Checador 2.0
  - Protocolo de trabajo (obligatorio)
  - Estado actual del sistema (resumen)
  - Historial de cambios recientes
    - 2026-03-31 · Correccion critica de clasificacion + reconciliacion masiva marzo (una sola vez)
    - 2026-03-30 · UX vista previa masiva: navegación completa de empleados
    - 2026-03-30 · Horario nocturno 18:00-07:00 + edición rápida RH desde vista previa
    - 2026-03-30 · Correccion de turnos nocturnos y modos invertidos (caso USERID 69)
    - 2026-03-30 · Preview masivo premium (master-detail + filtros + KPI)
    - 2026-03-30 · Vista previa de todos los empleados (sin Excel)
    - 2026-03-30 · Migración base a .NET 8
    - 2026-03-30 · Configuración centralizada (fuente única)
    - 2026-03-30 · Seguridad de configuración + corrección de textos corruptos
    - 2026-03-11 · Horarios + autorización de horas extra
    - 2026-03-11 · Incidente detectado en arranque
    - 2026-03-11 · Mejora de iniciador para evitar versión vieja involuntaria
    - 2026-03-11 · Mejora RH solicitada (horarios editables + DEPT/SHIFT + UI)
    - 2026-03-11 · Captura de horarios con AM/PM
    - 2026-03-11 · Fix UI: campos que parecían “no guardados”
    - Nota operativa de bitácora
    - 2026-03-11 · Ajuste reporte Excel semanal (campos de encabezado y resumen)
    - 2026-03-11 · Hotfix de compilación post-ajuste Excel
    - 2026-03-11 · Fix funcional plantilla (Horario + Total semanal)
    - 2026-03-11 · Modo servidor 24/7 con tarea programada
    - 2026-03-11 · Sincronizador con interfaz gráfica (pedido RH/operación)
    - 2026-03-11 · Arquitectura desacoplada: complemento externo SyncAgent
    - 2026-03-11 · Validación estricta de comandos del checador (InOutMode)

    - 2026-03-11 · Hardening Pervasive: corrida controlada en simulación + switch seguro por fases
    - 2026-03-11 · Plantilla operativa Go/No-Go (Fase 0 -> Fase 1)
    - 2026-03-11 · Scripts seguros para cambio de número y/o metadatos por rango
    - 2026-03-12 · Admin SQL: cambio de número por rango con contraseña y validación previa
    - 2026-03-18 · Ajuste operativo espejo Pervasive (habilitado + salida intermedia)
    - 2026-03-18 · Corrección anti-ruido: no cerrar salida por marcas demasiado cercanas
    - 2026-03-18 · Actualización correctiva automática de registros Pervasive mal cerrados
    - 2026-03-18 · Corrección también en MSSQL (PunchLogs)
    - 2026-03-18 · Nombres completos en SQL/reportes y nombres cortos solo en checador
- Checklist rápido antes de reportar “no aparece cambio”
- Plantilla de registro para próximos cambios
    - 2026-03-19 · Espejo Pervasive por segmentos (salida/entrada intermedia)
    - 2026-03-31 · Auditoria full-detail todos los USERID (marzo 2026) + endurecimiento diurno
    - 2026-03-31 · Ajuste puntual por marcas invertidas reportadas (USERID 10, 69, 120, 165)
    - 2026-03-31 · Normalizacion automatica m1->m0 + validaciones preventivas de alto impacto
    - 2026-03-31 · Vista previa masiva UX/UI + correccion de alcance de empleados
    - 2026-03-31 · Generacion de reportes con seleccion PDF o Excel
    - 2026-03-31 · Ajuste de firma: mostrar nombre del trabajador sin romper espacio de firma
    - 2026-03-20 · Turno nocturno cruzando fecha (preview MSSQL)
    - 2026-03-20 · Soporte de plantillas Excel con macros (.xlsm)
    - 2026-03-20 · Regla nocturna solo para horario nocturno (evitar impacto global)
    - 2026-03-26 · Sistema base de estilos de botones (Desktop)
    - 2026-03-26 · Propagacion global de tema entre formularios (ThemeManager)

- 2026-03-26 · Reemplazo de emoji por Segoe MDL2 en header (Form1)
- 2026-03-26 · Rediseño completo de KpiSummaryForm (tarjetas KPI)
- 2026-03-26 · Jerarquia visual en botones de acciones (Form1)
- 2026-03-19 · Descarga/AutoSync con correccion automatica de datos malos
- 2026-03-19 · Turno nocturno (Horario 3) corregido para cruce de medianoche
- 2026-03-19 · Ajuste MSSQL para preview nocturno (sin tocar Pervasive)

AI Cache · Checador 2.0

> Bitácora viva para **NO perder contexto**, evitar cambios improvisados y documentar errores/aprendizajes.
>
> Regla operativa: **antes de hacer cualquier cambio nuevo, leer este archivo completo; al terminar, actualizarlo.**

Protocolo de trabajo (obligatorio)

1. Leer este documento.
2. Confirmar el objetivo exacto del cambio.
3. Revisar archivos impactados.
4. Aplicar cambios mínimos y verificables.
5. Validar (build/errores/flujo UI).
6. Registrar aquí:
    - Qué se cambió
    - Por qué
    - Resultado
    - Errores y aprendizaje

Estado actual del sistema (resumen)

- App principal: `src/Checador.Desktop` (WinForms .NET 8 x86).
- Sync SQL incremental operativo (`Employees`, `PunchLogs`, `SyncState`).

- Espejo Pervasive activo con preview y reglas `INSERT/UPDATE/SKIP`.
- Admin SQL simplificado a una tabla principal + bitácora + preview.
- Detección intermedias soportada con `InOutMode` cuando está disponible.

Historial de cambios recientes

2026-03-31 · Correccion critica de clasificacion + reconciliacion masiva marzo (una sola vez)

Incidencia operativa reportada

- Se detectaron errores masivos de clasificacion en marzo:
  - entradas de 07:00/08:00 tomadas como salida,
  - salidas de 16:36/17:36 tomadas como entrada,
  - salida/entrada de comida cruzadas con entrada/salida de jornada,
  - inconsistencias entre preview MSSQL y espejo Pervasive.

Cambios de logica aplicados

- `AttendanceProcessor` reforzado con heuristicas cronologicas para modos invertidos:
  - fallback de entrada por cronologia cuando `InOutMode` llega invertido,
  - fallback de salida por cronologia cuando no existe cierre por modo,
  - inferencia controlada en dias `out-only` solo con evidencia fuerte (span >= 240 min),
  - prioridad a marca temprana sin modo cuando la entrada por modo aparece demasiado tarde (gap >= 180 min),
  - dias incompletos sin rellenar horas falsas `00:00`.
- `PervasiveAttendanceMirrorService` alineado a las mismas reglas para no divergir entre calculo y escritura.

Rutina operativa excepcional agregada

- ConsoleApp:
  - `--reconcile-march-2026-once`:

        - fuerza reconciliacion historica de marzo 2026 por lotes de dias (respeta candados de volumen),
        - ejecuta auditoria posterior de IDs reportados,
        - ejecuta auditoria mensual global de calidad.
    - `--audit-march-2026-only`:
        - corre solo auditorias (sin reescritura masiva).
- Script raiz creado:
    - `RECALCULAR-MARZO-2026-UNA-VEZ.cmd`
    - incluye confirmacion interactiva y overrides temporales de entorno para habilitar Pervasive solo en la corrida excepcional.

Ejecucion real realizada (marzo 2026)

- Corrida masiva inicial:
    - Insertados: 271
    - Actualizados: 521
    - Omitidos: 32
- Corrida tecnica final tras ajuste residual:
    - Insertados: 0
    - Actualizados: 797
    - Omitidos: 32

Resultado de auditoria posterior (IDs reportados)

- USERID 23: sospechosos = 0
- USERID 24: sospechosos = 0
- USERID 81: sospechosos = 0
- USERID 120: sospechosos = 0
- USERID 163: sospechosos = 0
- USERID 165: sospechosos = 0
- USERID 170: sospechosos = 4 (marcas atipicas de origen en madrugada/tarde)

Resultado mensual global despues de correccion

- Empleados auditados: 41
- Dias procesados: 723
- Incidencias: 244
- Dias sospechosos: 18
- Empleados con sospechosos: 9

Aprendizaje / nota operativa

- El problema principal fue inversion/ruido de `InOutMode` en origen del reloj, no solo de UI.
- La reconciliacion masiva de marzo se considera operacion excepcional y ya se dejo empaquetada para uso controlado.
- Los residuales actuales se concentran en pocos empleados con patron de marcas atipico (ej. madrugada), y requieren validacion RH caso por caso.

2026-03-30 · UX vista previa masiva: navegación completa de empleados

Qué se hizo

- Se mejoró el panel izquierdo de `AllEmployeesPreviewForm` para evitar pérdida de contexto al recorrer listas largas:
  - contador visible `Mostrando X/Y`
  - botón `Limpiar filtros`
  - navegación rápida con `<` y `>`
  - salto directo por `USERID` con Enter/botón `Ir`
  - scrollbar vertical explícito y ayuda visual actualizada.

Resultado esperado

- El usuario puede recorrer toda la lista sin quedarse en un tramo parcial (por ejemplo hasta USERID 161).
- Mayor claridad de cuántos empleados están visibles vs total.
- Flujo más profesional para análisis operativo diario.

2026-03-30 · Horario nocturno 18:00-07:00 + edición rápida RH desde vista previa

Qué se hizo

- Se normalizó `Horario 3` a entrada `18:00` y salida `07:00`:
  - defaults en código (`SqlDeviceSyncService`, `SyncAdminForm`, `AttendanceProcessingOptions`, `AppConfiguration`, `appsettings`).

    - validación y actualización directa en SQL para garantizar consistencia inmediata.
- Se validaron empleados nocturnos reales (`StartTime > EndTime`): `102`, `170`, `177`.
- Se aplicó ajuste de datos para USERID `69`: asignado a `Horario 3` con extras autorizadas y jornada estándar `9.60`.
- Se mejoró `AllEmployeesPreviewForm` para edición rápida y práctica:
    - combo de horario
    - checkbox de autorización de horas extra
    - botón `Guardar RH` en el panel de detalle
    - guardado directo a SQL vía `RenameEmployeeAsync` conservando demás datos.

Resultado esperado

- Turnos nocturnos quedan con ventana correcta 18:00-07:00.
- RH puede ajustar horario y autorización de extras sin salir de la vista previa masiva.
- Flujo más operativo y profesional para corregir configuraciones al momento.

2026-03-30 · Correccion de turnos nocturnos y modos invertidos (caso USERID 69)

Validacion realizada

- Se reviso SQL real de empleados/horarios y se detecto:
    - USERID 69 actualmente esta en `Horario 1` (07:00-16:36), no en horario nocturno.
    - Sus marcas del periodo traen `InOutMode` invertido (`1` en la primera marca y `0` en la segunda), lo que generaba incidencia falsa y salida en columna intermedia.
- Empleados detectados con horario nocturno real (`StartTime > EndTime`): `102`, `170`, `177`.

Que se corrigio en codigo

- `AttendanceProcessor` ahora aplica fallback cronologico cuando detecta inversion de modos (entrada/salida), evitando clasificaciones falsas por modo invertido.

- Se agrego fallback para tratar una unica salida `modo 2` como salida final cuando no hay marcas posteriores, evitando que se etiquete como comida por error.
- Horas extra se calculan en base a jornada estandar (`StandardShiftHours`) para todos los turnos cuando el empleado autoriza extras.
- `SqlAttendanceDataSource` ahora deriva `IsNightShift` desde `WorkSchedules` (`StartTime > EndTime`) y lo inyecta en `PunchRecord` para que el procesamiento nocturno no dependa solo de un codigo fijo.

Resultado esperado

- Se reducen incidencias falsas por modo invertido en dias con 2 marcas.
- Nocturnos se identifican por definicion real del horario.
- Horas extra respetan el umbral de jornada estandar configurado por empleado.

2026-03-30 · Preview masivo premium (master-detail + filtros + KPI)

Qué se hizo

- `AllEmployeesPreviewForm` se rediseñó a modo **master-detail**:
  - lista de empleados a la izquierda
  - detalle diario del empleado seleccionado a la derecha
  - sin abrir/cerrar ventana por cada revisión
- Se agregaron KPI globales en cabecera (empleados, horas, extra, incidencias).
- Se agregó filtro rápido por estado (`TODOS`, `OK`, `REVISAR`, `SIN_REGISTRO`).
- Se reforzó legibilidad:
  - semáforo por estado en filas
  - columnas clave congeladas (`USERID`, `Empleado`, `Fecha`)
  - tooltip en observaciones para no perder detalle por ancho de columna.

Resultado esperado

- Revisión masiva más rápida y más clara para RH/supervisión.
- Menos fricción al navegar empleado por empleado.
- Mejora visual de nivel ejecutivo sin cambiar la lógica de cálculo existente.

2026-03-30 · Vista previa de todos los empleados (sin Excel)

Qué se hizo

- Se agregó flujo de vista previa masiva en `Form1` cuando está activo `Generar todos los empleados`.
- El botón `Vista previa` ahora procesa a todos y muestra `KPI Summary` sin crear archivos Excel.
- Se agregó método dedicado en `AttendanceReportOrchestrator` para calcular resumen masivo sin generación de reportes.

Resultado esperado

- Operación puede validar horas/empleados de toda la plantilla antes de generar reportes.
- Se evita tiempo/costo de generar archivos cuando solo se quiere revisar consistencia.

Ajuste UI

- Se agregó pantalla `AllEmployeesPreviewForm` con tabla de empleados (filtro, estado, horas, incidencias).
- Doble clic o botón `Ver empleado` abre la vista detallada de ese empleado con el formato actual de `ReportPreviewForm`.
- Esto permite revisar caso por caso (por ejemplo empleado 14) sin generar Excel.

2026-03-30 · Migración base a .NET 8

Qué se hizo

- Todos los proyectos de la solución se migraron de `net6.0-windows` a `net8.0-windows`.
- Scripts operativos (`INICIAR-CHECADOR.cmd`, `INICIAR-SYNCAGENT.cmd`, `SINCRONIZAR-CONFIG.cmd`) se actualizaron a rutas `net8.0-windows`.
- Validación de runtime en `StartupRequirementsValidator` se elevó a requerir `.NET 8+`.
- Documentación principal se alineó a `.NET 8`.

Resultado esperado

- Se elimina el riesgo de soporte por framework EOL de .NET 6.
- El arranque y sincronización usan la misma ruta de salida en net8.

- Menos ruido operativo por advertencias de fin de soporte.

2026-03-30 · Configuración centralizada (fuente única)

Qué se hizo

- Se creó `SINCRONIZAR-CONFIG.cmd` en raíz para copiar `appsettings.json` fuente hacia rutas `bin/Debug` y `bin/Release` cuando existan.
- `INICIAR-CHECADOR.cmd` ahora invoca ese sincronizador al iniciar.
- `INICIAR-SYNCAGENT.cmd` ahora invoca ese sincronizador antes de abrir el agente.
- `README.md` se actualizó para indicar claramente qué archivos se deben editar y cuáles no.

Resultado esperado

- El equipo edita solo configuración fuente en `src/.../appsettings.json`.
- Se reduce el error operativo de modificar por accidente archivos dentro de `bin`.

2026-03-30 · Seguridad de configuración + corrección de textos corruptos

Qué se hizo

- `Checador.ConsoleApp`, `Checador.Desktop` y `Checador.SyncAgent` ahora cargan también:
  - `appsettings.local.json` (opcional)
  - variables de entorno con prefijo `CHECADOR_`
- `SINCRONIZAR-CONFIG.cmd` ahora sincroniza también `appsettings.local.json` cuando existe.
- Se corrigieron mensajes con mojibake en `SqlDeviceSyncService` y texto de incidencia en `AttendanceProcessor`.
- Se actualizó este documento para reflejar el estado real actual de candados Pervasive.

Resultado esperado

- Menor exposición de credenciales en archivos fuente compartidos.
- Mensajes operativos legibles y sin texto corrupto.
- Bitácora alineada con configuración efectiva de producción local.

2026-03-11 · Horarios + autorización de horas extra

Qué se hizo

- Se extendió `Employees` con:
  - `WorkScheduleCode`
  - `OvertimeAuthorized`
  - `StandardShiftHours`
- Se actualizó renombrar en Admin SQL para guardar también:
  - Horario (1/2/3)
  - Checkbox `Autorizo horas extra`
- Se conectó el datasource de reportes para traer esos campos desde SQL.
- Se ajustó cálculo de extras:
  - Si **no** está autorizado: extra = `0.00`
  - Si está autorizado: extra sobre jornada estándar del empleado.
- Se agregó modo consola `--daemon-sync` para autosincronización continua.

Por qué

- Requisito de negocio: extras sólo para personal autorizado y configuración por empleado.

Resultado esperado

- En el diálogo de renombrar deben verse controles de jornada/autorización.
- En reportes, extras condicionadas por autorización.

2026-03-11 · Incidente detectado en arranque

Síntoma

- Mensaje: `App abierta: se omite compilacion para evitar bloqueo de DLL`.
- El usuario no ve cambios nuevos en UI.

Causa raíz

- Se abrió una versión binaria previa porque no se recompiló con la app abierta.

Acción correctiva

- `INICIAR-CHECADOR.cmd` ahora muestra aviso explícito:
  - que abrirá versión anterior sin cambios recientes
  - que se debe cerrar Checador y re-ejecutar para aplicar cambios
- Se imprime timestamp de la última compilación del `.exe` Release.

Lección

- Si no hay recompilación, no esperar cambios visuales nuevos.
- Si `SyncAdminForm.cs` tiene hora más reciente que `bin\\Release\\...\\Checador.Desktop.exe`, el UI mostrará comportamiento viejo.

2026-03-11 · Mejora de iniciador para evitar versión vieja involuntaria

Qué se hizo

- `INICIAR-CHECADOR.cmd` ahora, si detecta app abierta, pregunta:
  - `S` = cerrar app y recompilar
  - `C` = continuar sin recompilar

Resultado esperado

- Menos confusión: no se abre binario viejo por accidente cuando hay cambios pendientes.

2026-03-11 · Mejora RH solicitada (horarios editables + DEPT/SHIFT + UI)

Qué se hizo

- Se creó catálogo SQL editable `WorkSchedules` (códigos 1/2/3) con:
  - nombre de horario

    - hora entrada
    - hora salida
    - horas estándar
    - activo/inactivo
- En Admin SQL se agregó botón `🗓 Horarios` para abrir editor visual y guardar cambios.
- En `Renombrar`, el combo de horario ahora muestra contexto completo (ej. `1 - Horario 1 (08:00-17:36, 9.6h)`).
- Se robusteció carga de `DEPT/SHIFT`:
    - si el valor existente no está en catálogo, se muestra como `Actual: <valor>` en lugar de perderse en `(sin seleccionar)`.
- Se amplió y modernizó el diálogo de `Renombrar` (más grande, mejor espaciado, botones visibles).

Problema que resuelve

- RH ya puede entender y mantener qué significa cada horario sin adivinar.
- Evita recapturar DEPT/SHIFT cuando existan valores heredados o no estándar.
- Evita botones tapados por tamaño/DPI.

2026-03-11 · Captura de horarios con AM/PM

Qué se hizo

- En `🗓 Horarios`, la captura cambió de `HH:mm` a formato amigable:
    - campo de hora (`hh:mm`)
    - selector `AM/PM`
- Al guardar, el sistema convierte automáticamente a 24h para persistir en SQL.

Resultado esperado

- RH ya no necesita pensar en formato militar; captura directa tipo 08:00 AM / 05:36 PM.

2026-03-11 · Fix UI: campos que parecían “no guardados”

Síntoma reportado

- En `Renombrar`, a veces volvía a salir `(sin seleccionar)` en Horario/DEPT/SHIFT aunque acababan de guardarse.

Causa probable

- El formulario precargaba datos desde la grilla visible (estado potencialmente desfasado), no desde lectura directa de SQL por `UserId`.

Corrección aplicada

- Se agregó `GetEmployeeAdminSnapshotAsync(userId)` en `SqlDeviceSyncService`.
- `SyncAdminForm` ahora usa snapshot directo desde SQL al abrir `Renombrar` (fallback a grilla sólo si no existe snapshot).
- Se reforzó fallback para preferir valores **no vacíos** entre snapshot y grilla (`PreferNonEmpty`) y se agregó log de precarga en bitácora UI.
- Si un horario guardado no existe en catálogo de horarios, se muestra como `Actual (no catalogado)` en combo.

Resultado esperado

- Lo que se guarda en SQL se refleja al reabrir diálogo, sin pedir recaptura innecesaria.

Nota operativa de bitácora

- Mensaje `Espejo Pervasive con error (no bloqueante...)` aplica al espejo de asistencias (sincronización de marcas), **no** al guardado de datos RH (nombre/depto/puesto/DEPT/SHIFT/horario) en SQL.

2026-03-11 · Ajuste reporte Excel semanal (campos de encabezado y resumen)

Cambios aplicados

- Se enriqueció `ReportRequest` / `ReportTemplateContext` con datos de horario y tipo de jornada.
- Desde `Form1`, al enriquecer con SQL, ahora se envía:
  - `WorkScheduleCode`

    - `WorkScheduleDisplay` (ej. `Horario 1 (07:00 AM a 05:36 PM)`)
    - `WorkShiftType` (`Diurna` para 1/2, `Nocturna` para 3)
- `ClosedXmlReportGenerator` ahora escribe en plantilla:
    - `Horario:`
    - `Tipo de jornada:`
    - marca visual best-effort en `Diurna/Nocturna` con `X` cuando el formato lo permite.
- Se robusteció escritura de totales semanales (`Total de horas laboradas`, `Total de horas extra`) para etiquetas combinadas/merges.

Resultado esperado

- El Excel llena encabezado de horario/tipo jornada y bloque resumen semanal con totales.

2026-03-11 · Hotfix de compilación post-ajuste Excel

Incidente

- Falló build con:
    - `CS1061` en `AttendanceReportOrchestrator` (uso incorrecto de `.Value` en nullable)
    - `CS0103/CS0128` en `ClosedXmlReportGenerator` (líneas corruptas inyectadas en `WriteTotals`).

Corrección

- Se simplificó `derivedScheduleCode` para conservar tipo nullable correcto.
- Se limpió `WriteTotals` y se dejó helper `GetValueCellToRight` correctamente definido a nivel clase.
- Se aplicó helper también para totales de plantilla con celdas combinadas.

2026-03-11 · Fix funcional plantilla (Horario + Total semanal)

Síntoma

- En algunos formatos, `Horario` quedaba como `00:00 a 00:00 Hrs`.
- `Total de horas laboradas en la semana` no siempre se llenaba.

Corrección

- `Form1` ahora envía `WorkScheduleDisplay` en formato 24h: `HH:mm a HH:mm Hrs`.
- `ClosedXmlReportGenerator` usa selección más inteligente de celda destino en la misma fila (`FindBestValueCellOnRow`) para que `Horario:` sobrescriba el placeholder correcto.
- Se ampliaron variantes de búsqueda para etiqueta de total semanal:
  - `totaldehoraslaboradasporsemana`
  - `totaldehoraslaboradasenlasemana`
  - `totalhoraslaboradasenlasemana`

2026-03-11 · Modo servidor 24/7 con tarea programada

Archivos creados

- `INICIAR-CHECADOR-SERVIDOR.cmd`: arranca `Checador.ConsoleApp.exe --daemon-sync` y sincroniza `appsettings.json` de consola.
- `INSTALAR-TAREA-SERVIDOR.cmd`: crea tarea `Checador\ChecadorServerAutoSync` (`ONSTART`, `SYSTEM`, `HIGHEST`, delay 30s) y lanza prueba.
- `DESINSTALAR-TAREA-SERVIDOR.cmd`: elimina la tarea.

Nota operativa

- La instalación de tarea requiere privilegios de Administrador. Si sale `Acceso denegado`, ejecutar `INSTALAR-TAREA-SERVIDOR.cmd` con clic derecho → "Ejecutar como administrador".

2026-03-11 · Sincronizador con interfaz gráfica (pedido RH/operación)

Implementado

- Nueva ventana `AutoSyncControlForm` (desde `Admin SQL`) con:

    - botón `Activar/Detener AutoSync`
    - selector de intervalo en minutos
    - mini bitácora de eventos en tiempo real
- Integración en `SyncAdminForm` con botón `📄 AutoSync GUI`.

Comportamiento

- Al activar, corre un ciclo inicial inmediato y luego ciclos programados.
- Evita solapamiento de ejecuciones (`_busy` guard).

2026-03-11 · Arquitectura desacoplada: complemento externo SyncAgent

Decisión

- El autosync GUI se movió a un complemento externo independiente de `Checador.Desktop`.
- `Checador` consume resultados desde SQL (y espejo Pervasive), no necesita ejecutar el motor de sync local.

Nuevo proyecto

- `src/Checador.SyncAgent` (`Checador.SyncAgent.exe`)
    - UI propia de autosync (activar/detener, intervalo, bitácora)
    - usa `SqlDeviceSyncService` y `ZkTecoDirectAttendanceDataSource`

Scripts operativos

- `INICIAR-SYNCAGENT.cmd`
- `INSTALAR-TAREA-SYNCAGENT.cmd`
- `DESINSTALAR-TAREA-SYNCAGENT.cmd`

2026-03-11 · Validación estricta de comandos del checador (InOutMode)

Problema reportado

- Salida intermedia física se estaba interpretando como salida normal final.

Regla aplicada en `AttendanceProcessor`

- `0` = Entrada inicial
- `1` = Salida final
- `2` = Salida intermedia
- `3` = Regreso intermedio

Comportamiento nuevo

- Si hay modo `2` pero no hay `1`, se marca `INCIDENCIA` y no se cierra jornada como continua.
- Solo `modo 1` cierra jornada como salida final.
- Estado por modo:
  - sin intermedia -> `JORNADA_CONTINUA`
  - intermedia + regreso -> `OK`
  - intermedia sin regreso/final -> `INCIDENCIA`

2026-03-11 · Hardening Pervasive: corrida controlada en simulación + switch seguro por fases

Objetivo

- Validar espejo Pervasive con riesgo mínimo antes de habilitar escritura real.
- Evitar daño histórico o cierres incorrectos por salidas intermedias.

Candados de referencia (modo seguro recomendado)

- `SimulationMode = true`
- `OnlyCurrentDayWrites = true`
- `CloseOnlyWithFinalOutMode = true`
- `MaxRowsPerRun = 200`

Estado actual (2026-03-30)

- `src/Checador.SyncAgent/appsettings.json` → `SimulationMode=false`, `OnlyCurrentDayWrites=false`, `CloseOnlyWithFinalOutMode=false`.
- `src/Checador.Desktop/appsettings.json` → `SimulationMode=false`, `OnlyCurrentDayWrites=false`, `CloseOnlyWithFinalOutMode=false`.

- `src/Checador.ConsoleApp/appsettings.json` → espejo deshabilitado (`Enabled=false`).

Corrida de validación controlada (solo simulación)

1. Iniciar `SyncAgent` con `INICIAR-SYNCAGENT.cmd`.
2. Abrir UI de `SyncAgent` y ejecutar `🔄 Sync ahora` (o activar autosync).
3. En `Checador.Desktop` → `Admin SQL` → `👁 Preview Pervasive`.
4. Revisar resumen y notas:
   - Esperado: acciones `INSERT/UPDATE/SKIP` sin escritura real (por simulación).
   - Si hay candados actuando: aparecerán `SKIP_GUARD` o `ABORT_GUARD` con causa en `Notas`.
5. Repetir 2–3 ciclos en distinto momento del día para confirmar estabilidad.

Criterios para pasar a fase de escritura real

- No aparecen `ABORT_GUARD` inesperados (si aparece, ajustar `MaxRowsPerRun` con criterio).
- La mayoría de filas candidatas son coherentes con operación del día.
- Casos con salida intermedia (`InOutMode=2`) quedan protegidos cuando falta salida final (`modo 1`).
- Sin errores de conexión Pervasive en bitácora.

Switch seguro por fases (recomendado)

- **Fase 0 (referencia):** simulación total.
  - `SimulationMode=true`
  - `OnlyCurrentDayWrites=true`
  - `CloseOnlyWithFinalOutMode=true`
- **Fase 1 (piloto acotado, 1 día):** habilitar escritura solo día actual.
  - Cambiar únicamente `SimulationMode=false`.
  - Mantener los otros 3 candados exactamente igual.
  - Monitorear preview + bitácora por jornada completa.
- **Fase 2 (estabilización, 3–5 días):** mantener igual y revisar incidentes.
  - No relajar `OnlyCurrentDayWrites` ni `CloseOnlyWithFinalOutMode` aún.
- **Fase 3 (solo si negocio lo exige):** evaluar ampliar alcance.
  - Cualquier relajación debe ser un cambio explícito, documentado y reversible.

Rollback inmediato (si algo no cuadra)

- Regresar `SimulationMode=true`.
- Reiniciar `SyncAgent`.
- Verificar que preview vuelva a comportamiento de simulación.

Observabilidad agregada

- `SyncAdminForm` y `PervasivePreviewForm` ya reportan también:
  - `SKIP_GUARD`
  - `ABORT_GUARD`

Esto permite distinguir omisiones normales (`SKIP`) de bloqueos por candado (seguridad).

2026-03-11 · Plantilla operativa Go/No-Go (Fase 0 -> Fase 1)

Usar esta plantilla al cierre del día para decidir si se permite pasar a `SimulationMode=false` por 1 jornada piloto.

Regla de decisión (semáforo)

- 🟢 **GO**: todos los criterios críticos en verde.
- 🟡 **GO CONDICIONAL**: 1 criterio no crítico en amarillo + plan de mitigación escrito.
- 🔴 **NO-GO**: cualquier criterio crítico en rojo.

Criterios críticos (deben quedar en verde)

1. `ABORT_GUARD` = 0 en todas las revisiones del día.
2. Sin errores de conexión Pervasive en bitácora operativa.
3. Casos con `InOutMode=2` sin `1` terminan en `SKIP_GUARD` (no cierre indebido).
4. Preview coherente con operación real del día (sin picos anómalos de filas).

Criterios no críticos (aceptables con mitigación)

1. Variaciones esperadas en `SKIP` por empleados incompletos.
2. Ajuste menor de `MaxRowsPerRun` por volumen operativo del día.

Formato de evaluación diaria (copiar/pegar)

```
### YYYY-MM-DD · Decisión Go/No-Go Pervasive (F0 -> F1)

- Responsable:
- Turno/ventana evaluada:
- Build/versión ejecutada:

#### Evidencia recolectada

- Corridas de preview revisadas: N
- INSERT:
- UPDATE:
- SKIP:
- SKIP_GUARD:
- ABORT_GUARD:
- Errores Pervasive (sí/no + detalle):

#### Validación de criterios críticos

- [ ] ABORT_GUARD = 0
- [ ] Sin errores de conexión Pervasive
- [ ] Intermedias sin final protegidas (SKIP_GUARD)
- [ ] Preview coherente con operación del día

#### Resultado

- Semáforo: 🟢 GO / 🟡 GO CONDICIONAL / 🔴 NO-GO
- Decisión: (mantener Fase 0 / pasar a Fase 1)
- Justificación corta:
- Mitigación/acciones para siguiente día:
```

Cambio controlado para Fase 1 (solo si GO)

1. Cambiar únicamente `SimulationMode=false` en:
   - `src/Checador.SyncAgent/appsettings.json`
   - `src/Checador.Desktop/appsettings.json` (si se usa Desktop para sync directo)
2. No tocar:
   - `OnlyCurrentDayWrites` (debe permanecer `true`)
   - `CloseOnlyWithFinalOutMode` (debe permanecer `true`)
   - `MaxRowsPerRun` (mantener valor validado)
3. Reiniciar `SyncAgent` y registrar hora exacta del cambio.

Rollback de emergencia (si aparece riesgo)

- Síntomas de rollback inmediato:
  - `ABORT_GUARD` inesperado repetido
  - comportamiento no coherente en preview

    - error recurrente de conexión/escritura
- Acción inmediata:
    1. Regresar `SimulationMode=true`
    2. Reiniciar `SyncAgent`
    3. Registrar incidente y volver a Fase 0 por 24h

2026-03-11 · Scripts seguros para cambio de número y/o metadatos por rango

Necesidad operativa

- Cambiar `USERID` y/o actualizar `Nombre/DEPT/SHIFT` sin tocar horarios de entradas/salidas.
- Aplicar solo en ventana acotada (ej. últimos 2 días), no histórico completo.

Artefactos creados

- `docs/sql/CAMBIO-EMPLEADO-SQLSERVER-RANGO.sql`
    - transaccional
    - simulación con `@ApplyChanges = 0`
    - manejo de conflicto por índice único `(UserId, CheckTime)`
    - actualización de `PunchLogs` solo en rango + upsert de `Employees` destino
- `docs/sql/CAMBIO-EMPLEADO-PERVASIVE-RANGO.sql`
    - plantilla para `TIME_ATTENDANCE`
    - update solo de `EMPLOYEE/NAME/DEPT/SHIFT` por `CHARGE_DATE`
    - sin tocar `IN_TS_MIN/OUT_TS_MIN` ni `DATE/TIME IN/OUT`
- `docs/sql/GUIA-CAMBIO-EMPLEADO-RANGO.md`
    - orden operativo seguro (detener sync -> simular -> aplicar -> validar)

Regla operativa reforzada

- Primero SQL Server en simulación y validación.
- Luego SQL Server real.
- Luego Pervasive.
- Al final reactivar `SyncAgent` y validar preview.

2026-03-12 · Admin SQL: cambio de número por rango con contraseña y validación previa

Implementado en UI (`SyncAdminForm`)

- Nuevo botón: 🔐 `Cambio # empleado (rango)`.
- Protección por contraseña obligatoria para ejecutar:
  - `Innovaxpw*d1`
- Formulario de captura con:
  - USERID origen (seleccionado en grilla)
  - USERID destino
  - Fecha inicio / Fecha fin
  - Nombre (opcional)
  - DEPT / SHIFT (Pervasive)

Validación previa obligatoria (antes de aplicar)

- Muestra resumen SQL + Pervasive con:
  - filas origen y destino
  - conflictos detectados
  - **lista de días exactos** que se modificarán en el rango
- El usuario confirma explícitamente antes de ejecutar.

Lógica backend agregada

- `SqlDeviceSyncService`
  - `PreviewEmployeeReassignByDateRangeAsync(...)`
  - `ReassignEmployeeByDateRangeAsync(...)`
- `PervasiveAttendanceMirrorService`
  - `PreviewEmployeeRemapByDateRangeAsync(...)`
  - `ApplyEmployeeRemapByDateRangeAsync(...)`

Reglas de seguridad aplicadas

- Solo opera dentro del rango indicado.
- En SQL no toca `CheckTime` ni `InOutMode`.
- En Pervasive no toca `IN_TS_MIN`, `OUT_TS_MIN`, `DATE/TIME IN/OUT`.
- Omite conflictos para proteger unicidad (`(UserId, CheckTime)` en SQL y `EMPLOYEE+CHARGE_DATE` en Pervasive) cuando aplica.

2026-03-18 · Ajuste operativo espejo Pervasive (habilitado + salida intermedia)

Síntoma reportado

- La bitácora de sync mostraba: `Espejo Pervasive deshabilitado`.
- Se requería confirmar uso de base `GLOBALPIN`.
- Se detectó que no se estaban considerando salidas intermedias para cierre en espejo.

Cambio aplicado

- `src/Checador.Desktop/appsettings.json`
  - `PervasiveSync.Enabled = true`
  - `PervasiveSync.ConnectionString` verificada con `DBQ=GLOBALPIN`
  - `PervasiveSync.OnlyCurrentDayWrites = false` (permite reconciliar días recientes, no solo hoy)
  - `PervasiveSync.CloseOnlyWithFinalOutMode = false` (permite tomar salida intermedia como salida elegible cuando aplique)
- `src/Checador.SyncAgent/appsettings.json`
  - `PervasiveSync.ConnectionString` verificada con `DBQ=GLOBALPIN`
  - `PervasiveSync.CloseOnlyWithFinalOutMode = false`

Resultado esperado

- En próximas corridas de `Download Attendance Logs` ya no debe aparecer `Espejo Pervasive deshabilitado`.
- El espejo Pervasive debe considerar salidas intermedias bajo la regla configurada.

2026-03-18 · Corrección anti-ruido: no cerrar salida por marcas demasiado cercanas

Problema observado

- En algunos casos, dos marcas casi al mismo minuto (ej. segundos de diferencia) se estaban tomando como entrada + salida y cerraban mal `TIME_ATTENDANCE`.
- Esto también contaminaba la vista previa operativa.

Corrección aplicada

- Se agregó candado configurable `MinimumOutGapMinutes` en `PervasiveSyncOptions`.

- Regla nueva en espejo Pervasive:
  - si la salida candidata está a menos de `MinimumOutGapMinutes` de la entrada, se ignora para cierre.
- Se aplicó en:
  - corrida normal de espejo
  - preview Pervasive
  - fallback rápido de espejo
- Se propagó config en:
  - `src/Checador.Desktop/appsettings.json`
  - `src/Checador.SyncAgent/appsettings.json`
  - `src/Checador.ConsoleApp/appsettings.json` con valor inicial: `MinimumOutGapMinutes = 1`.

Recuperación de datos ya contaminados

- Script de limpieza controlada por empleado/rango:
  - `docs/sql/REPARAR-PERVASIVE-RANGO.sql`
- Flujo recomendado:
  1. respaldar filas objetivo (SELECT)
  2. borrar solo empleado/rango afectado (DELETE)
  3. ejecutar de nuevo `Download Attendance Logs` para reconstrucción correcta.

2026-03-18 · Actualización correctiva automática de registros Pervasive mal cerrados

Necesidad

- Evitar depender siempre de borrado manual cuando ya existen filas mal cerradas en `TIME_ATTENDANCE`.
- Mostrar en bitácora cuántas filas se corrigieron automáticamente.

Implementado

- El espejo ahora detecta filas existentes con salida inválida por cercanía (`gap < MinimumOutGapMinutes`) y limpia salida (`OUT_TS_MIN/DATE_OUT/TIME_OUT`) de forma automática.

- Se agregó contador explícito en mensaje de corrida:
  - `actualizados=...`
  - `actualizados_correctivo=...`
- En Preview Pervasive se agregó acción:
  - `UPDATE_CORRECTIVE`

Resultado esperado

- La bitácora mostrará algo como:
  - `Insertados=10, actualizados=40, actualizados_correctivo=40, omitidos=...`

2026-03-18 · Corrección también en MSSQL (PunchLogs)

Necesidad

- El ajuste no debía quedarse solo en Pervasive; también se requería normalizar datos erróneos en SQL Server.

Implementado

- En cada corrida de sync se ejecuta limpieza correctiva en `dbo.PunchLogs` dentro de la misma transacción:
  - elimina marcas casi duplicadas por usuario/día/mismo modo cuando el gap es menor a 60 segundos.
- Se agregó contador en mensaje de bitácora SQL:
  - `sql_actualizados_correctivo=...`

Resultado esperado

- Menos ruido histórico en SQL para preview/reportes.
- Bitácora de sync con contador explícito de corrección SQL en cada corrida.

2026-03-18 · Nombres completos en SQL/reportes y nombres cortos solo en checador

Problema

- Algunos nombres llegan truncados por límite del dispositivo y eso no debe contaminar SQL ni reportes.

Implementado

- `Employees.EmployeeName` en SQL se preserva como fuente canónica:
  - sync desde reloj ya no sobreescribe nombres completos existentes con nombre corto del dispositivo.
- Inserción de `PunchLogs` usa preferencia de nombre canónico de `Employees` al guardar marcas.
- Consultas de SQL para reportes/historial/preview prefieren `Employees.EmployeeName` (`COALESCE`) sobre nombre de marca.
- En renombrado:
  - SQL guarda nombre completo.
  - al checador se envía versión recortada segura (24 chars) solo para compatibilidad del dispositivo.

Resultado esperado

- Reportes y vistas SQL con nombre completo.
- Checador conserva el nombre corto permitido por firmware sin afectar datos empresariales.

Checklist rápido antes de reportar “no aparece cambio”

- ☐ ¿Checador estaba cerrado al iniciar?
- ☐ ¿El `.cmd` compiló sin error?
- ☐ ¿Se abrió `bin\\Release\\net8.0-windows\\Checador.Desktop.exe`?
- ☐ ¿La fecha/hora de compilación del EXE es reciente?

Plantilla de registro para próximos cambios

```
### YYYY-MM-DD · Título corto

- Qué se hizo:
- Archivos tocados:
- Validación:
- Error encontrado:
- Lección/aprendizaje:
- Pendiente siguiente:
```

2026-03-19 · Espejo Pervasive por segmentos (salida/entrada intermedia)

Problema observado

- En `TIME_ATTENDANCE` se estaban pisando filas cuando habia mas de un tramo el mismo dia.
- La logica previa actualizaba por `EMPLOYEE + CHARGE_DATE`, lo que no distingue segmentos.
- Resultado: casos con salida/entrada intermedia podian quedar en una sola fila o con tramo incompleto.

Correccion aplicada

- `src/Checador.Infrastructure.Access/PervasiveAttendanceMirrorService.cs`
  - Se introdujo construccion de segmentos por dia:
    - Segmento 1: `entrada -> salida intermedia` (si existe modo 2 y regreso modo 3).
    - Segmento 2: `entrada intermedia -> salida final` (o abierto sin salida).
  - El upsert del espejo ahora se hace por llave de entrada (`DATE_IN/TIME_IN`) dentro del dia, no solo por `EMPLOYEE+CHARGE_DATE`.
  - Se actualizaron las 3 rutas para usar el mismo criterio:
    - `MirrorSnapshotAsync` (ruta normal)
    - `BuildPreviewAsync` (preview operativo)
    - `MirrorSnapshotFastFallbackAsync` (contingencia rapida)

Impacto en AutoSync

- El cambio queda aplicado automaticamente en AutoSync porque:

    - `DownloadAttendanceLogsAsync` usa `MirrorSnapshotAsync`.
    - si hay contingencia por timeout, usa `MirrorSnapshotFastFallbackAsync`.
- Por tanto, Desktop GUI, SyncAgent y corrida automatica quedan alineados con la misma regla.

Validacion tecnica

- `dotnet build Checador.sln` completado sin errores.

2026-03-31 · Auditoria full-detail todos los USERID (marzo 2026) + endurecimiento diurno

Objetivo

- Revisar todos los USERID dia por dia en marzo 2026.
- Endurecer reglas para que no se vuelva a tomar una entrada espuria de madrugada en turno diurno cuando existe alternativa matutina valida.

Cambios aplicados

- `src/Checador.Application/AttendanceProcessor.cs`
    - Nueva ventana minima diurna para entrada (`DayShiftEarliestEntryBoundary = 04:00`).
    - En el guard temporal de entrada (cuando la entrada detectada cae despues de mediodia), ahora se busca candidato matutino preferente:
        - primero modo 0,
        - luego modos de entrada,
        - luego modo nulo,
        - y por ultimo la primera marca en ventana.
- `src/Checador.Infrastructure.Access/PervasiveAttendanceMirrorService.cs`
    - Se alineo exactamente la misma regla de candidato matutino para el espejo Pervasive.

Validacion ejecutada

- Build:
    - `dotnet build Checador.sln` OK.

- Auditoria detallada completa:
  - Comando: `dotnet run --project src/Checador.ConsoleApp/Checador.ConsoleApp.csproj -- --audit-march-2026-full-detail`
  - Archivo: `logs/auditoria-marzo-2026-detalle-20260331-092058.csv`
  - Resultado: `filas=1302`, `filas_sospechosas=196`, `empleados_con_sospecha=39`.

Lectura de resultados (minuciosa)

- La mayor parte de sospechosos son faltas reales de marca (no inversion de clasificacion):
  - `MISSING_EXIT = 132`
  - `MISSING_ENTRY = 32`
- Anomalias fuertes de clasificacion/incongruencia temporal: `32` filas totales.
  - Archivo enfocado: `logs/auditoria-marzo-2026-detalle-20260331-092058-hard.csv`
  - Checks mas frecuentes en hard:
    - `RAW_MODE_INVERTED = 12`
    - `EXIT_BEFORE_NOON = 7`
    - `BREAK_OUT_NEAR_EXIT = 6`
    - `ENTRY_AFTER_NOON = 4`

Caso representativo corregido por la regla nueva

- USERID 170, fecha 2026-03-12:
  - antes quedaba entrada de madrugada (01:12),
  - ahora se ajusta a entrada matutina valida (07:09), con salida 19:00.

2026-03-31 · Ajuste puntual por marcas invertidas reportadas (USERID 10, 69, 120, 165)

Reporte recibido

- USERID 10 (dias 25 y 26): entrada aparecia como salida.
- USERID 69 (dias 25 y 26): par invertido de modos en jornada completa.
- USERID 120 (dia 26): entrada aparecia como salida.

- USERID 165 (dia 26): entrada aparecia como salida.

Correccion aplicada

- `src/Checador.Application/AttendanceProcessor.cs`
  - `CanInferEntryFromOutOnlyDay(...)` ahora permite inferir entrada para dia de una sola marca en modo salida (1/2), solo si:
    - no hay contexto nocturno (`IsNightShift` o `WorkScheduleCode = 3`),
    - la marca es matutina (< 12:00).
  - Resultado: en incompletos con una sola marca matutina invertida, se reporta `Entrada` y `Salida` vacia (MISSING_EXIT), en lugar de `Salida` sin `Entrada`.
- `src/Checador.Infrastructure.Access/PervasiveAttendanceMirrorService.cs`
  - Se alineo la misma inferencia de entrada para el espejo Pervasive.
  - `ResolveOutPunchCandidate(...)` se endurecio para no elegir como salida la misma marca usada como entrada (requiere marca posterior).

Validacion de casos reportados

- Auditoria generada: `logs/auditoria-marzo-2026-detalle-20260331-093733.csv`.
- USERID 10:
  - 2026-03-25: `Entrada=08:26`, `Salida=-`, `Checks=MISSING_EXIT`.
  - 2026-03-26: `Entrada=08:07`, `Salida=-`, `Checks=MISSING_EXIT`.
- USERID 120:
  - 2026-03-26: `Entrada=08:01`, `Salida=-`, `Checks=MISSING_EXIT`.
- USERID 165:
  - 2026-03-26: `Entrada=08:02`, `Salida=-`, `Checks=MISSING_EXIT`.
- USERID 69:
  - 2026-03-25 y 2026-03-26 quedan con `Entrada/Salida` cronologicamente correctas.

Ejecucion operativa

- Se ejecuto nuevamente reconciliacion de marzo para reflejar el ajuste en Pervasive:
  - `insertados=12`, `actualizados=789`, `omitidos=22`.

2026-03-31 · Normalizacion automatica m1->m0 + validaciones preventivas de alto impacto

Solicitud

- Convertir automaticamente pares invertidos `m1 -> m0` (salida/entrada) a estatus normal cuando la cronologia sea claramente valida.
- Agregar validaciones adicionales para prevenir clasificaciones de alto impacto en el futuro.

Implementado

- `src/Checador.Application/AttendanceProcessor.cs`
  - Normalizacion explicita de par invertido valido (`m1 -> m0`) para jornada de 2 marcas:
    - ventana minima de jornada: `>= 240 min`.
    - ventana maxima diurna segura: `<= 1080 min`.
    - validacion diurna: entrada antes de mediodia y salida despues de mediodia.
  - Si se usa fallback cronologico y el par invertido cumple validacion, se conserva estatus normal (`JORNADA_CONTINUA`) en lugar de `INCIDENCIA`.
  - Se agrego guard de seguridad para fallback de salida cuando la ultima marca es modo de entrada: solo se acepta si pasa validacion fuerte de cronologia.
  - Se agrego guard de jornada diurna fuera de ventana para bloquear clasificaciones de alto impacto.
- `src/Checador.Infrastructure.Access/PervasiveAttendanceMirrorService.cs`
  - Mismas validaciones para cierre de salida inferida en espejo Pervasive.
  - Si la inversion no es concluyente, se descarta el cierre automatico y no se escribe salida dudosa.
- `src/Checador.ConsoleApp/Program.cs`
  - Auditoria detallada: `RAW_MODE_INVERTED` ya no se marca cuando el par `m1 -> m0` fue validado como cronologicamente correcto.

Validacion tecnica y operativa

- Build: `dotnet build Checador.sln` exitoso.
- Auditoria full-detail:
  - Archivo: `logs/auditoria-marzo-2026-detalle-20260331-094728.csv`
  - Resultado: `filas=1302`, `filas_sospechosas=186` (bajo de 196), `empleados_con_sospecha=39`.
- Reconciliacion operativa marzo ejecutada despues del ajuste:
  - `insertados=0`, `actualizados=801`, `omitidos=22`.
  - Auditoria final post-reconciliacion:
    - `logs/auditoria-marzo-2026-detalle-20260331-094820.csv`

Resultado puntual solicitado

- USERID 69 (2026-03-25 y 2026-03-26):
  - `Estatus=JORNADA_CONTINUA`, `Checks=OK`.
  - Se corrige el impacto de inversion `m1 -> m0` sin perder control en casos ambiguos.

2026-03-31 · Vista previa masiva UX/UI + correccion de alcance de empleados

Problema reportado

- En "Ver todos los empleados" la lista se quedaba visualmente en USERID 161.
- Acomodo de filtros/botones/tabla percibido como desordenado y poco usable.

Causa funcional corregida

- `src/Checador.Application/AttendanceReportOrchestrator.cs`
  - `BuildPreviewForAllAsync(...)` ahora no depende solo de `GetEmployeesAsync()`.
  - Se fusiona catalogo base + USERID detectados en marcas del rango.
  - Si un USERID tiene marcas pero no existe en catalogo, se agrega con nombre inferido de la marca (o `Empleado_{USERID}`).
  - Resultado: la vista previa masiva ya incluye tambien empleados fuera del catalogo tradicional (evita corte aparente en 161).

Mejora UX/UI aplicada

- `src/Checador.Desktop/AllEmployeesPreviewForm.cs`

- Panel izquierdo redisenado con `TableLayoutPanel` responsivo:
  - fila de busqueda/filtros/salto por USERID alineada,
  - barra de navegacion de empleados ordenada,
  - tabla principal en `Dock=Fill` sin tamanos fijos.
- `SplitContainer` ajustado para mejor proporciones (`SplitterDistance`, `Panel1MinSize`, `SplitterWidth`).
- Navegacion por teclado mejorada:
  - `Up/Down` mueve 1 empleado,
  - `PageUp/PageDown` mueve 5.
- Se elimino layout por coordenadas manuales que causaba mala adaptacion en resoluciones/DPI.

Validacion tecnica

- `dotnet build Checador.sln` exitoso despues de los cambios.

2026-03-31 · Generacion de reportes con seleccion PDF o Excel

Solicitud

- Al presionar generar, preguntar formato de salida: PDF o Excel.
- Generar archivo en el formato elegido.

Implementado

- `src/Checador.Domain/Models/ReportOutputFormat.cs`
  - Nuevo enum de formato: `Excel` / `Pdf`.
- `src/Checador.Domain/Interfaces/IReportGenerator.cs`
  - `GenerateWeeklyReportAsync(...)` ahora recibe `outputFormat`.
- `src/Checador.Application/AttendanceReportOrchestrator.cs`
  - Se propaga `outputFormat` en:
    - `GenerateAsync(...)`
    - `GenerateWithSummaryAsync(...)`
    - `GenerateForAllAsync(...)`
    - `GenerateForAllWithSummaryAsync(...)`
- `src/Checador.Infrastructure.Excel/ClosedXmlReportGenerator.cs`
  - Si formato es `Excel`, mantiene flujo actual con plantilla `.xlsx/.xlsm`.

    - Si formato es `Pdf`, primero genera el Excel real y luego exporta a PDF usando Microsoft Excel COM (`ExportAsFixedFormat`).
    - Esto preserva formato/plantilla para que el PDF sea visualmente igual al Excel generado.
- `src/Checador.Desktop/Form1.cs` + `src/Checador.Desktop/ReportFormatPickerDialog.cs`
    - Se reemplaza el `MessageBox` simple por dialogo UX/UI dedicado con tarjetas de opcion:
        - `Generar PDF`
        - `Generar Excel`
        - `Cancelar`
    - Se usa el formato elegido tanto para generacion individual como masiva.

Validacion tecnica

- `dotnet build Checador.sln` exitoso con la funcionalidad nueva.

2026-03-31 · Ajuste de firma: mostrar nombre del trabajador sin romper espacio de firma

Solicitud

- En el bloque `Nombre y firma del trabajador`, agregar el nombre del empleado sin romper el espacio visual de firma.

Implementado

- `src/Checador.Infrastructure.Excel/ClosedXmlReportGenerator.cs`
    - En `WriteDeclarationMetadata(...)` se prioriza escribir el nombre dentro de la celda etiqueta de firma, no en celdas lejanas.
    - Se agrega helper `TryWriteNameInsideDeclarationLabel(...)`:
        - limpia espacios duplicados,
        - trunca nombres demasiado largos,
        - aplica `ShrinkToFit` y `WrapText=false` para mantener la zona de firma estable.
    - Se conserva fallback existente por compatibilidad con otras plantillas.

Validacion tecnica

- `dotnet build Checador.sln` exitoso despues del ajuste.

2026-03-20 · Turno nocturno cruzando fecha (preview MSSQL)

Solicitud

- Si un empleado entra de noche (ej. 19:00) y sale en madrugada (ej. 03:00+), todo debe contarse en el mismo dia laboral.
- Corregir solo procesamiento MSSQL/preview (sin reescribir Pervasive historico).

Ajuste aplicado

- `src/Checador.Application/AttendanceProcessor.cs`
  - Se completo la asignacion de `workDate` por empleado antes de agrupar por dia.
  - Se agrego un arrastre contextual para madrugada:
    - si una marca cae antes de `NightShiftCarryToPreviousDayUntilHour` (default 12:00),
    - y existe marca de la noche anterior (>= `NightShiftStartHour`, default 19:00) dentro de ventana logica,
    - la marca de madrugada se asigna al dia laboral anterior.
  - Se agrego tolerancia de entrada anticipada (hasta 2 horas antes del inicio nocturno) para cubrir casos reales como 18:50 en turno de 19:00.
  - Cuando el `WorkScheduleCode` no llega como nocturno pero la sesion cruza medianoche con patron nocturno, se infiere turno nocturno para calcular extra despues de `NightShiftRegularEndHour` (03:00 por defecto).
  - Esto evita que entrada y salida nocturna queden separadas como dos dias `INCOMPLETO`.

Compatibilidad

- El ajuste impacta el calculo de preview/reporte (MSSQL) y no modifica la insercion historica ya existente en Pervasive.
- AutoSync mantiene la misma base de reglas al usar el mismo pipeline de descarga/correccion SQL.

Validacion tecnica

- `dotnet build src/Checador.Application/Checador.Application.csproj -c Release` exitoso.
- Build de solucion completa con error de copia por bloqueo de archivos (`Checador.Desktop` en ejecucion), no por error de codigo.

2026-03-20 · Soporte de plantillas Excel con macros (.xlsm)

Solicitud

- Aceptar plantilla con macros y generar reporte manteniendo macros.

Cambios

- `src/Checador.Infrastructure.Excel/ClosedXmlReportGenerator.cs`
  - El archivo de salida ahora usa extension dinamica:
    - plantilla `.xlsm` -> salida `.xlsm`
    - plantilla `.xlsx` -> salida `.xlsx`
  - Se agrego preservacion de proyecto VBA al guardar:
    - copia `xl/vbaProject.bin` (y firma si existe),
    - asegura relaciones en `xl/_rels/workbook.xml.rels`,
    - asegura tipos de contenido en `[Content_Types].xml`,
    - fuerza `workbook.xml` como `macroEnabled` cuando la salida es `.xlsm`.
- `src/Checador.Desktop/Form1.cs`
  - Selector de plantilla actualizado para aceptar `.xlsx` y `.xlsm`.
  - Mensaje de bloqueo de plantilla actualizado para ambos formatos.
- `README.md`
  - Documentacion de UI actualizada para indicar plantillas `.xlsx` o `.xlsm`.

Validacion tecnica

- `dotnet build src/Checador.ConsoleApp/Checador.ConsoleApp.csproj -c Release` correcto.
- `dotnet build src/Checador.Desktop/Checador.Desktop.csproj -c Release` llega a compilacion y falla solo al copiar DLLs porque la app Desktop estaba abierta (archivo bloqueado).

2026-03-20 · Regla nocturna solo para horario nocturno (evitar impacto global)

Incidencia

- La condicion nocturna se estaba aplicando a empleados sin horario nocturno, mezclando marcas entre dias en preview.

Ajuste aplicado

- `src/Checador.Application/AttendanceProcessor.cs`
  - Se elimino el arrastre heuristico para empleados sin horario nocturno.
  - Se elimino inferencia nocturna por patron horario para usuarios sin `WorkScheduleCode` nocturno.
  - El cruce de fecha y calculo nocturno vuelven a aplicar solo cuando `IsNightShiftPunch(...)` es verdadero (horario nocturno configurado).

Resultado esperado

- Empleados diurnos ya no heredan reglas de turno nocturno.
- La validacion especial de nocturno queda acotada al horario correspondiente.

Validacion tecnica

- `dotnet build src/Checador.Application/Checador.Application.csproj -c Release` correcto.
- `dotnet build src/Checador.ConsoleApp/Checador.ConsoleApp.csproj -c Release` correcto.

2026-03-26 · Sistema base de estilos de botones (Desktop)

Solicitud

- Crear clase base reutilizable para estilos de botones sin tocar formularios.

Cambios

- Nuevo archivo: `src/Checador.Desktop/ButtonStyles.cs`
  - Clase estatica `ButtonStyles` con metodos:

        - `ApplyPrimary(Button btn)`
        - `ApplySecondary(Button btn)`
        - `ApplyDanger(Button btn)`
    - Base comun aplicada en los 3:
        - `FlatStyle = Flat`
        - `Cursor = Hand`
        - `Font = Segoe UI 9f`
        - configuracion de `FlatAppearance` (borde + hover) segun variante.
    - Colores implementados segun especificacion:
        - Primary: `#0078D4` hover `#106EBE`
        - Secondary: borde/texto `#0078D4` hover `#EFF6FC`
        - Danger: borde/texto `#D83B01` hover `#FDF3F0`

Validacion tecnica

- `dotnet build src/Checador.Desktop/Checador.Desktop.csproj -c Release` correcto.

2026-03-26 · Propagacion global de tema entre formularios (ThemeManager)

Solicitud

- Propagar tema claro/oscuro fuera de `Form1` sin tocar logica de negocio ni controles individuales.

Cambios

- Nuevo archivo: `src/Checador.Desktop/ThemeManager.cs`
    - `CurrentTheme` (string: "Claro"/"Oscuro")
    - evento `ThemeChanged`
    - `Apply(string theme)` para persistir y notificar
    - `ApplyToForm(Form form)` para aplicar color base del formulario:
        - Claro: `BackColor=White`, `ForeColor=(30,30,30)`
        - Oscuro: `BackColor=(32,32,32)`, `ForeColor=White`
- `src/Checador.Desktop/Form1.cs`
    - En `cmbTheme_SelectedIndexChanged` se agrego:
        - `ThemeManager.Apply(cmbTheme.SelectedItem?.ToString() ?? "Claro");`

- Formularios suscritos y aplicacion inicial en `Load`:
  - `src/Checador.Desktop/SyncAdminForm.cs`
  - `src/Checador.Desktop/ReportPreviewForm.cs`
  - `src/Checador.Desktop/KpiSummaryForm.cs`
  - `src/Checador.Desktop/PervasivePreviewForm.cs`

Validacion tecnica

- `dotnet build src/Checador.Desktop/Checador.Desktop.csproj -c Release` correcto.

2026-03-26 · Reemplazo de emoji por Segoe MDL2 en header (Form1)

Solicitud

- Quitar emoji en labels/botones de `Form1` y usar glifo de Segoe para el icono del encabezado.

Cambios

- `src/Checador.Desktop/Form1.Designer.cs`
  - `lblAppGlyph.Font` actualizado a `Segoe MDL2 Assets, 24f`.
  - `lblAppGlyph.Text` actualizado a `\uE823` (reloj).
- Verificacion de botones:
  - `btnGenerate`, `btnPreview`, `btnOpenReport`, `btnTestConnection`, `btnAdminSql` ya estaban sin emoji, se conservaron solo con texto descriptivo.

Validacion tecnica

- Build intento de Desktop con fallo de copiado por DLL bloqueada (`Checador.Desktop` en ejecucion), sin errores de codigo en este cambio.

2026-03-26 · Rediseño completo de KpiSummaryForm (tarjetas KPI)

Solicitud

- Reemplazar resumen en TextBox por layout visual completo (header, tarjetas KPI, detalles y footer).

Implementado

- `src/Checador.Desktop/KpiSummaryForm.cs`
  - Refactor a clase parcial.
  - Se mantiene constructor compatible: `KpiSummaryForm(ReportRunSummary summary)` (sin cambios en Form1).
  - Se agregan propiedades publicas:
    - `EmpleadosProcesados`
    - `EmpleadosSinRegistro`
    - `HorasTotales`
    - `HorasExtra`
    - `CarpetaSalida`
    - `ResumenAdicional`
  - Dibujo de icono check verde en circulo usando `Graphics` en header.
  - Boton `Cerrar` mantiene `DialogResult.OK` y estilo `ButtonStyles.ApplyPrimary`.
- `src/Checador.Desktop/KpiSummaryForm.Designer.cs`
  - Nuevo layout (aprox 420x380):
    - Header con icono + titulo "Generación completada" + subtitulo dinamico.
    - Grid 2x2 de tarjetas KPI con colores por metrica:
      - procesados azul `#0078D4`
      - sin registro naranja `#CA5010`
      - horas trabajadas verde `#107C10`
      - horas extra morado `#5C2D91`
    - Separador horizontal `#E0E0E0`.
    - Seccion de detalles: reportes generados y carpeta de salida.
    - Footer con `FlowLayoutPanel` y boton `Cerrar`.
  - Propiedades de formulario:
    - `FormBorderStyle.FixedDialog`
    - `StartPosition.CenterParent`
    - `MinimizeBox = false`
    - `MaximizeBox = false`
    - `BackColor = White`

Validacion tecnica

- `dotnet build src/Checador.Desktop/Checador.Desktop.csproj -c Release` correcto.

2026-03-26 · Jerarquia visual en botones de acciones (Form1)

Solicitud

- Aplicar estilos de `ButtonStyles` solo a los 5 botones principales del `flpActionButtons`.

Cambios

- `src/Checador.Desktop/Form1.cs`
  - En `InitializeDefaults()` se aplico:
    - `ButtonStyles.ApplyPrimary(btnGenerate)`
    - `ButtonStyles.ApplySecondary(btnPreview)`
    - `ButtonStyles.ApplySecondary(btnOpenReport)`
    - `ButtonStyles.ApplySecondary(btnTestConnection)`
    - `ButtonStyles.ApplySecondary(btnAdminSql)`

Alcance

- Sin cambios de logica/eventos ni otros controles.

Validacion tecnica

- `dotnet build src/Checador.Desktop/Checador.Desktop.csproj -c Release` correcto.

2026-03-19 · Descarga/AutoSync con correccion automatica de datos malos

Solicitud operativa

- Que al usar `Download Attendance Logs` se corrijan datos malos automaticamente.
- Que `AutoSync` use exactamente la misma logica.

Implementado

- `src/Checador.Infrastructure.Access/SqlDeviceSyncService.cs`
  - Se reforzo limpieza correctiva SQL en cada corrida con ventana minima semanal:
    - `weeklyCorrectiveFrom = hoy - 7 dias`
    - si hubo inserciones antiguas en la corrida, usa la fecha mas antigua entre ambos criterios.
  - El espejo Pervasive ahora siempre toma lote de reconciliacion desde SQL (ventana correctiva), no solo marcas recien insertadas.
  - En contingencia por timeout, el fallback ahora usa el lote de reconciliacion calculado (y no depende solo de `insertedPunches`).

Impacto en AutoSync

- `AutoSync` (Desktop GUI y SyncAgent) llama internamente `DownloadAttendanceLogsAsync`, por lo tanto hereda esta correccion automatica sin pasos extra.

Resultado esperado

- Cada descarga/sync intenta actualizar datos inconsistentes recientes en SQL + Pervasive de forma continua.
- Menor necesidad de correcciones manuales despues de cada corrida.

2026-03-19 · Turno nocturno (Horario 3) corregido para cruce de medianoche

Problema reportado

- Marcas de turno nocturno (entrada ~19:00 y salida madrugada) se estaban partiendo por fecha calendario.
- Eso generaba dias mezclados y calculos irreales de horas/extra.

Implementado

- `src/Checador.Application/AttendanceProcessor.cs`
  - Agrupacion por `workDate` para horario nocturno:
    - si `WorkScheduleCode == 3` y la marca es antes de la hora de inicio nocturno, se asigna al dia laboral anterior.

    - Calculo de extra nocturna por regla de corte:
        - para horario 3, extra se calcula despues de `NightShiftRegularEndHour` (default 03:00).
- `src/Checador.Application/AttendanceProcessingOptions.cs`
    - Nuevas opciones:
        - `EnableNightShiftSupport`
        - `NightShiftScheduleCode`
        - `NightShiftStartHour`
        - `NightShiftRegularEndHour`
        - `NightShiftStandardHours`
- Se cablearon opciones en:
    - `src/Checador.Desktop/Program.cs`
    - `src/Checador.ConsoleApp/Program.cs`
    - `src/Checador.Desktop/AppConfiguration.cs`
    - `src/Checador.ConsoleApp/AppConfiguration.cs`
    - `src/Checador.Desktop/appsettings.json`
    - `src/Checador.ConsoleApp/appsettings.json`

Valores por defecto aplicados

- `NightShiftScheduleCode = 3`
- `NightShiftStartHour = 19`
- `NightShiftRegularEndHour = 3`
- `NightShiftStandardHours = 8`

Validacion tecnica

- `dotnet build Checador.sln` completado sin errores.

2026-03-19 · Ajuste MSSQL para preview nocturno (sin tocar Pervasive)

Solicitud

- Corregir registros mal interpretados en preview/reporte nocturno.
- Aplicar correccion sobre MSSQL y dejar intacto lo ya insertado en Pervasive.

Cambios aplicados (MSSQL + procesamiento)

- `src/Checador.Application/AttendanceProcessor.cs`
  - Agrupacion nocturna ajustada: ahora el arrastre al dia anterior solo aplica hasta `NightShiftCarryToPreviousDayUntilHour` (default 12), no hasta las 19:00.
  - Normalizacion por minuto en memoria para preview/reporte:
    - colapsa marcas repetidas del mismo minuto y prioriza modo de entrada sobre salida para evitar falsas salidas intermedias.
- `src/Checador.Infrastructure.Access/SqlDeviceSyncService.cs`
  - Limpieza correctiva MSSQL extendida:
    - sigue limpiando duplicados muy cercanos por mismo modo,
    - y ademas limpia colisiones del mismo minuto por usuario/dia priorizando modo valido.
  - Esta limpieza ocurre en cada `DownloadAttendanceLogsAsync`, por lo que tambien aplica en AutoSync.

Configuracion nueva

- `NightShiftCarryToPreviousDayUntilHour` (default 12)
  - agregado en opciones/config para Desktop y Console.

Importante

- No se modifico logica ni datos historicos de Pervasive en este ajuste.

Validacion tecnica

- `dotnet build Checador.sln` completado sin errores.